

Proyecto 2: Formulario AJAX con envío POST y respuesta del servidor

Objetivo del proyecto para el alumno

- Aprender a enviar datos de un formulario al servidor sin recargar la página usando **AJAX**.
- Procesar la **respuesta del servidor** y mostrarla dinámicamente en la página.
- Practicar la **comunicación cliente-servidor** con envío de datos en **formato JSON**.

Lo que se espera que haga el alumno:

1. Crear un formulario de contacto (nombre, email, mensaje).
2. Enviar los datos al servidor con **POST** usando Fetch y **JSON**.
3. Mostrar la respuesta del servidor en la página.
4. Agregar validaciones y mensajes dinámicos.

1. Código del servidor con Express (Node.js)

1.1 Crear proyecto

Si no tienes un proyecto Node ya, puedes usar:

```
mkdir proyecto_formulario
cd proyecto_formulario
npm init -y
npm install express cors
```

1.2 Servidor (**server.js**)

```
const express = require('express');
const cors = require('cors');
const app = express();

// Middleware
app.use(cors());
app.use(express.json()); // Para procesar JSON en el body

// Ruta para recibir formulario
app.post('/api/contacto', (req, res) => {
  const { nombre, email, mensaje } = req.body;

  // Validaciones simples
  if (!nombre || !email || !mensaje) {
    return res.status(400).json({ success: false, msg: 'Todos los campos son obligatorios' });
  }
});
```

```

    // Simulación de guardado o procesamiento
    console.log(`Mensaje recibido de ${nombre} (${email}):
    ${mensaje}`);

    // Respuesta al cliente
    res.json({ success: true, msg: `¡Gracias ${nombre}, tu mensaje ha
    sido recibido!` });
  });

// Iniciar servidor
const PORT = 3001;
app.listen(PORT, () => {
  console.log(`Servidor escuchando en http://localhost:${PORT}`);
});

```

- **Puerto:** 3001
- **Ruta:** POST <http://localhost:3001/api/contacto>
- Devuelve un **JSON** con la respuesta.

2. Código del cliente HTML + Bootstrap + AJAX

2.1 HTML con Bootstrap

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Formulario de Contacto AJAX</title>
  <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">
  <h1>Formulario de Contacto</h1>

  <form id="formContacto" class="mb-3">
    <div class="mb-2">
      <label for="nombre" class="form-label">Nombre</label>
      <input type="text" id="nombre" class="form-control"
name="nombre">
    </div>
    <div class="mb-2">
      <label for="email" class="form-label">Email</label>
      <input type="email" id="email" class="form-control"
name="email">
    </div>
    <div class="mb-2">
      <label for="mensaje" class="form-label">Mensaje</label>
      <textarea id="mensaje" class="form-control"
name="mensaje"></textarea>
    </div>
    <button type="submit" class="btn btn-primary">Enviar</button>
  </form>

  <div id="respuesta"></div>

```

```
    <script src="script.js"></script>
</body>
</html>
```

2.2 JavaScript (**script.js**) con Fetch

```
const form = document.getElementById('formContacto');
const respuesta = document.getElementById('respuesta');

form.addEventListener('submit', function(e) {
    e.preventDefault(); // Evitar recarga

    const datos = {
        nombre: document.getElementById('nombre').value,
        email: document.getElementById('email').value,
        mensaje: document.getElementById('mensaje').value
    };

    fetch('http://localhost:3001/api/contacto', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(datos)
    })
    .then(res => res.json())
    .then(data => {
        if(data.success){
            respuesta.innerHTML = `<div class="alert alert-
success">${data.msg}</div>`;
            form.reset();
        } else {
            respuesta.innerHTML = `<div class="alert alert-
danger">${data.msg}</div>`;
        }
    })
    .catch(err => {
        respuesta.innerHTML = `<div class="alert alert-danger">Error
al enviar el formulario</div>`;
        console.error(err);
    });
});
```

3. Funcionamiento del proyecto

1. Iniciar el servidor con:

```
node server.js
```

2. Abrir `index.html` en el navegador.
 3. Completar el formulario y hacer clic en **Enviar**.
 4. El servidor recibe los datos, valida, y devuelve un JSON con la respuesta.
 5. La página muestra la respuesta **sin recargar**.
-

4. Ejercicios propuestos para los alumnos

1. **Validación adicional:** hacer que el campo email sea obligatorio y con formato correcto antes de enviar.
2. **Campo opcional:** agregar un teléfono opcional y mostrarlo en la respuesta si se envía.
3. **Simular retraso:** en el servidor, usar `setTimeout` para retrasar la respuesta 2 segundos y mostrar un mensaje "Enviando..." mientras tanto.
4. **Lista de mensajes:** guardar los mensajes en un array en el servidor y crear una ruta `GET /api/contactos` para que se puedan mostrar todos los mensajes enviados.
5. **Diseño Bootstrap:** usar tarjetas (`card`) para mostrar la respuesta en lugar de alertas.